

INFORME TECNICO DEL PROYECTO AEROSHOP

Plataforma de comercio electronico (E-commerce) full-stack.

Fecha del informe: Junio 2026

Este documento describe la arquitectura, los archivos y la logica de negocio de todo el codigo contenido en la carpeta Proyecto.

Stack tecnologico:

- Frontend: HTML5, CSS3, JavaScript vanilla (SPA con hash routing)
- Backend: Node.js + Express 4
- Base de datos: SQLite3
- Autenticacion: JWT (jsonwebtoken) + bcryptjs
- Dependencias: cors, express, sqlite3, bcryptjs, jsonwebtoken

1. ARQUITECTURA GENERAL

El proyecto sigue un patron monolitico desacoplado:

[Navegador] <--HTTP--> [Express API + Static Files] <--SQL--> [SQLite]

El servidor Express (backend/app.js) cumple dos funciones:

- a) Servir la API REST bajo el prefijo /api
- b) Servir los archivos estaticos del frontend desde frontend/

El frontend es una SPA (Single Page Application) que usa el hash de la URL (#home, #login, #checkout, etc.) para navegar sin recargar.

Separacion de responsabilidades en el backend:

routes/ -> Definen endpoints HTTP y middleware
controllers/-> Logica de negocio y respuestas JSON
services/ -> Servicios externos simulados (pagos, notificaciones)
middleware/ -> Autenticacion JWT y control de roles
config/ -> Conexion y helpers de base de datos
db/ -> Esquema SQL y datos semilla

Modo demostracion offline: api.js incluye un motor MockDB que emula toda la API en localStorage si el backend no esta disponible.

2. ESTRUCTURA DE CARPETAS

Proyecto/

index.html -> Copia/entrada alternativa del frontend

package.json -> Scripts raiz: npm start, npm test

informe_proyecto.pdf-> Este informe

frontend/

index.html -> Pagina principal de la SPA

css/styles.css -> Estilos globales (tema oscuro, glassmorphism)
js/api.js -> Cliente HTTP + simulacion offline
js/auth.js -> Gestion de sesion en localStorage
js/views.js -> Plantillas HTML de cada vista
js/app.js -> Router, carrito, notificaciones, eventos

backend/
server.js -> Punto de entrada, puerto 5000, graceful shutdown
app.js -> Configuracion Express y montaje de rutas
package.json -> Dependencias del backend
config/database.js -> Conexion SQLite, dbQuery async, auto-seed
db/schema.sql -> DDL de 8 tablas
db/seed.sql -> Datos iniciales (productos, usuarios, cupones)
controllers/ -> 6 controladores de dominio
routes/ -> 7 routers Express
middleware/ -> authMiddleware.js (protect, isAdmin)
services/ -> paymentService.js, notifyService.js
tests/ -> integration.test.js (pruebas E2E)

scripts/
generate_report_pdf.js -> Generador de este PDF

.vscode/launch.json -> Configuracion de depuracion VS Code

3. BASE DE DATOS (schema.sql)

SQLite con claves foraneas activadas (PRAGMA foreign_keys = ON).

Tabla users:

id, email (UNIQUE), password_hash, full_name, role (client|admin),
auth_provider (email|google|apple), created_at

Tabla categories:

id, name, description, slug (UNIQUE), created_at

Tabla products:

id, name, description, price, stock, image_url, category_id (FK)

Tabla coupons:

id, code (UNIQUE), discount_type (percentage|fixed),
discount_value, min_order_value, active, expires_at

Tabla orders:

id, user_id (FK), status, total_amount, discount_amount,
final_amount, coupon_id (FK), payment_method, payment_status,
tracking_number (UNIQUE), created_at

Estados: pending, paid, shipped, delivered, cancelled,
refund_requested, refunded

Tabla order_items:

id, order_id (FK CASCADE), product_id (FK RESTRICT),
quantity, unit_price, total_price

Tabla refunds:

id, order_id (FK), reason, status (pending|approved|rejected),
requested_at, processed_at, admin_notes

Tabla notifications:

id, user_id (FK CASCADE), message, is_read, type, created_at

4. DATOS SEMILLA (seed.sql)

4 categorias: Tecnologia, Moda, Hogar y Cocina, Deportes y Fitness.
10 productos de ejemplo con precios en pesos colombianos (COP).

Usuarios predefinidos:

Admin: admin@ecommerce.com / admin123 (rol: admin)

Cliente: cliente@ecommerce.com / client123 (rol: client)

Cupones de prueba:

DESCUENTO10 -> 10% descuento, minimo \$50.000

SALE50K -> \$50.000 fijos, minimo \$300.000

BIENVENIDA -> 15% descuento, sin minimo

CADUCADO -> Cupon inactivo para pruebas de error

database.js inicializa automaticamente:

1. Ejecuta schema.sql al arrancar
2. Si categories esta vacia, ejecuta seed.sql
3. Expone dbQuery con metodos run/get/all/exec en Promises

5. BACKEND - server.js y app.js

server.js:

- Importa app y la conexion db
- Escucha en PORT (env o 5000 por defecto)
- Maneja SIGTERM/SIGINT para cierre graceful:
cierra servidor HTTP y luego la conexion SQLite

app.js:

- Middleware: cors(), express.json()
- express.static sirve frontend/ en la raiz
- Monta routers bajo /api/*
- GET /api/health -> estado del servicio
- Middleware global de errores -> JSON 500

Rutas montadas:

/api/auth -> authRoutes
/api/products -> productRoutes
/api/categories -> categoryRoutes
/api/orders -> orderRoutes (protegidas)
/api/refunds -> refundRoutes (protegidas)
/api/notifications -> notificationRoutes (protegidas)
/api/admin -> adminRoutes (admin only)

6. BACKEND - AUTENTICACION

authMiddleware.js:

protect: Lee header Authorization: Bearer <token>,
verifica JWT con JWT_SECRET, adjunta req.user
isAdmin: Verifica req.user.role === admin
JWT_SECRET: variable de entorno o valor por defecto de desarrollo

authController.js:

register: Valida email/password/fullName, hashea con bcrypt,
crea usuario rol client, retorna JWT 24h
login: Compara hash bcrypt, rechaza cuentas OAuth-only
oauth: Simula Google/Apple; crea o vincula usuario existente

authRoutes.js:

POST /register, POST /login, POST /oauth (publicas)

Flujo frontend (auth.js):

setSession(token, user) -> localStorage aero_token, aero_user
logout() -> limpia storage y dispara evento authChange
isAuthenticated(), getUser(), isAdmin() -> helpers de sesion

7. BACKEND - PRODUCTOS Y CATEGORIAS

productController.js:

getProducts: JOIN products+categories, filtros ?category y ?search
getProductById: Producto individual con category_name
getCategories: Lista ordenada por nombre

productRoutes.js:

GET / -> listado con filtros
GET /:id -> detalle (publicas)

categoryRoutes.js:

GET / -> todas las categorias (publica)

adminController.js (CRUD productos, solo admin):

createProduct: Valida categoria, inserta producto
updateProduct: Actualizacion parcial de campos
deleteProduct: DELETE con manejo de FK RESTRICT

(409 si el producto tiene pedidos historicos)

adminRoutes productos:

POST /admin/products

PUT /admin/products/:id

DELETE /admin/products/:id

8. BACKEND - PEDIDOS Y CUPONES

orderController.js - validateCoupon:

Verifica existencia, activo, fecha expiracion, monto minimo

orderController.js - createOrder (transaccional):

1. BEGIN TRANSACTION

2. Valida stock de cada item del carrito

3. Calcula total y aplica cupon si es valido

4. INSERT orders (pending) + order_items

5. Decrementa stock de productos

6. COMMIT

7. Llama PaymentService.processPayment()

8a. Exito: status=paid, notifica al usuario

8b. Fallo: ROLLBACK stock, status=cancelled, notifica error

orderController.js - consultas:

getMyOrders: Historial del usuario autenticado

getOrderById: Detalle + items + refund; solo dueno o admin

orderRoutes.js (todas con protect):

POST /coupons/validate

POST /

GET /my-orders

GET /:id

9. BACKEND - REEMBOLSOS Y ADMIN

refundController.js - requestRefund (cliente):

Valida pedido paid/delivered, sin solicitud previa

INSERT refund pending, UPDATE order refund_requested

Notifica cliente y administradores

refundController.js - processRefund (admin):

approve: PaymentService.processRefund(), repone stock,

status order=refunded, notifica cliente

reject: status refund=rejected, order vuelve a paid

adminController.js adicional:

getUsers: Lista usuarios sin password_hash

getOrders: Todos los pedidos con datos del cliente

updateOrderStatus: Cambia estado y notifica (shipped, etc.)

adminRoutes (protect + isAdmin):

GET /users, GET /orders, GET /refunds

POST /refunds/:id/process

PUT /orders/:id/status

10. BACKEND - SERVICIOS

paymentService.js (simulacion de pasarela):

processPayment(method, amount, details):

card: Requiere cardNumber, expiryDate, cvv

Falla si cardNumber termina en 9999

pse: Requiere bankName, userDocument

Falla si bankName contiene "rechazo"

wallet: Requiere phoneNumber, otpCode

Falla si otpCode === 0000

Latencia simulada: 500ms

processRefund(trackingNumber, amount):

Siempre exitoso, latencia 400ms, genera refundId

notifyService.js:

create(userId, message, type) -> INSERT notifications

getByUser(userId) -> SELECT ordenado por fecha DESC

markAsRead(id, userId) -> UPDATE is_read=1 con validacion

notificationController.js + notificationRoutes:

GET /notifications (protegida)

PUT /notifications/:id/read (protegida)

11. FRONTEND - index.html

Estructura HTML estatica de la SPA AeroShop:

<header class="navbar">

Logo, links: Catalogo, Mis Pedidos (cliente), Admin (admin)

Campana de notificaciones con dropdown

Boton carrito con badge de cantidad

Boton login / avatar de usuario

<main id="app">

Contenedor donde el router inyecta vistas dinamicamente

<div id="cart-drawer-overlay">

Drawer lateral: items, cupones, subtotal, checkout

<div id="toast-container">

Notificaciones toast temporales

Scripts cargados en orden:

api.js -> auth.js -> views.js -> app.js

Recursos externos: Google Fonts (Outfit, Inter), Font Awesome 6

12. FRONTEND - api.js

Cliente HTTP centralizado (API_BASE_URL = localhost:5000/api).

Metodos: get, post, put, delete con headers JSON + Bearer token.
handleResponse: Parsea JSON, en 401 limpia sesion y dispara authChange.

Fallback offline (useLocalMock):

Si fetch falla, activa MockDB en localStorage y muestra banner.

MockDB.init() crea categorias, productos, usuarios, cupones, pedidos vacios, notificaciones de bienvenida.

mockRequest() replica endpoints del backend:

Productos, categorias, auth, cupones, pedidos, reembolsos, notificaciones y todas las rutas admin.

Incluye logica de pago fallido (9999, OTP 0000, banco rechazo) y control de integridad al eliminar productos con pedidos.

Latencia simulada: 300ms en modo mock.

13. FRONTEND - views.js

Motor de plantillas HTML (objeto Views):

formatCurrency(value) -> formato COP es-CO

Vistas publicas:

Home(products, categories) -> grid con busqueda y filtro

ProductDetail(product) -> ficha con selector de cantidad

Login() / Register() -> formularios + botones OAuth simulados

Vistas autenticadas (cliente):

Checkout() -> acordeon 2 pasos: envio + pago (card/pse/wallet)

CheckoutSuccess() -> confirmacion con tracking number

Orders() -> tabla historial de pedidos

OrderDetail() -> timeline, items, formulario reembolso

Vista admin:

Admin(tab, data) -> sidebar con 4 pestanas:

products: tabla CRUD con botones editar/eliminar

orders: selector de estado por pedido

refunds: aprobar/rechazar solicitudes pendientes
users: listado de cuentas registradas

14. FRONTEND - app.js

Controlador principal de la SPA:

AppState global:

cart[], appliedCoupon, activeCheckoutStep, currentAdminTab

CartManager:

Persiste carrito en localStorage (aero_cart)

addToCart, removeFromCart, updateQuantity, clear

calculateTotals() con descuentos de cupon

updateUI() renderiza drawer lateral

NotificationManager:

Polling cada 30s de /notifications si hay sesion

renderNotifications(), markAsRead()

router(): Navegacion por hash

/ #product/:id / #login / #register / #checkout

#orders / #orders/:id / #admin/:tab

Protege rutas privadas redirigiendo a #login

setupHomeEvents, setupDetailEvents, setupLoginEvents, etc.:

Vinculan eventos DOM despues de renderizar cada vista

Modales dinamicos: openProductModal(), openRefundModal()

setupGlobalUI(): carrito, cupones, notificaciones, navbar

updateNavbarAuthUI(): muestra/oculta links segun rol

Inicializacion en DOMContentLoaded: cart, UI, router, hashchange

15. FRONTEND - styles.css y auth.js

styles.css (no listado linea a linea):

Define variables CSS (--primary, --danger, --success, etc.)

Tema oscuro premium con efecto glass (backdrop-filter)

Componentes: navbar, cards, product-grid, cart-drawer,

checkout-steps, admin-grid, modals, toasts, tablas, badges

Animaciones: spinner de carga, transiciones del drawer

Responsive: grid adaptativo para catalogo y checkout

auth.js:

Modulo minimo de 59 lineas para gestion de sesion.

Almacena JWT en aero_token y datos de usuario en aero_user.

Dispara evento personalizado authChange al login/logout

para que app.js actualice navbar y re-renderice rutas.

index.html (raiz Proyecto/):

Duplicado funcional de frontend/index.html con rutas css/js relativas a la carpeta frontend (css/, js/).

16. FLUJOS PRINCIPALES DE NEGOCIO

Flujo de compra:

1. Usuario navega catalogo, agrega productos al carrito
2. Opcionalmente aplica cupon (requiere login)
3. Checkout: datos envio + metodo de pago
4. POST /api/orders -> reserva stock, procesa pago
5. Exito: carrito vacio, pantalla confirmacion, notificacion

Flujo de reembolso:

1. Cliente ve pedido paid/delivered en #orders/:id
2. Enví^a motivo -> POST /api/refunds
3. Admin revisa en #admin/refunds
4. Aprueba -> reversa pago simulada + stock repuesto

Flujo admin productos:

1. Admin accede a #admin/products
2. CRUD via modales -> POST/PUT/DELETE /api/admin/products

Estados de pedido (ciclo de vida):

pending -> paid -> shipped -> delivered

paid -> refund_requested -> refunded (via reembolso)

pending -> cancelled (pago fallido o admin)

17. API REST - RESUMEN DE ENDPOINTS

PUBLICOS:

GET /api/health

POST /api/auth/register | /login | /oauth

GET /api/products, /api/products/:id

GET /api/categories

PROTEGIDOS (Bearer JWT):

POST /api/orders/coupons/validate

POST /api/orders

GET /api/orders/my-orders, /api/orders/:id

POST /api/refunds

GET /api/notifications

PUT /api/notifications/:id/read

SOLO ADMIN (JWT + role admin):

GET /api/admin/users | /orders | /refunds

POST /api/admin/products | /refunds/:id/process
PUT /api/admin/products/:id | /orders/:id/status
DELETE /api/admin/products/:id

Codigos de error habituales:

400 validacion, 401 sin token, 403 sin permisos,
404 no encontrado, 409 conflicto FK, 500 error servidor

18. PRUEBAS DE INTEGRACION

backend/tests/integration.test.js:

Levanta servidor en puerto 5099 y ejecuta pruebas E2E con fetch.

Modulos probados:

1. Auth: registro, duplicados, login cliente y admin
2. Productos: listado, filtros, detalle por ID
3. Pedidos: validacion cupon, creacion con pago exitoso
4. Pedidos: pago fallido (tarjeta 9999) y restauracion stock
5. Reembolsos: solicitud cliente, aprobacion admin
6. Admin: CRUD productos, cambio estado pedido
7. Notificaciones: listado y marcar como leida

Ejecutar: npm test (desde raiz o backend/)

Escenarios de pago para pruebas manuales:

Tarjeta terminada en 9999 -> pago rechazado

OTP wallet 0000 -> pago rechazado

Banco "Banco de Rechazo" en PSE -> pago rechazado

19. COMO EJECUTAR EL PROYECTO

Requisitos: Node.js 18+ instalado.

Pasos:

1. cd Proyecto/backend && npm install
2. cd .. && npm start
(ejecuta node backend/server.js en puerto 5000)
3. Abrir navegador en http://localhost:5000

Alternativa sin backend:

Abrir frontend/index.html directamente en el navegador.

api.js activara modo demostracion local automaticamente.

Variables de entorno opcionales:

PORT=5000 -> puerto del servidor

JWT_SECRET=... -> clave para firmar tokens JWT

NODE_ENV=production -> modo de ejecucion

Depuracion VS Code:

.vscode/launch.json configura arranque del server.js

20. CONCLUSION

AeroShop es una plataforma E-commerce educativa y funcional que demuestra patrones profesionales de desarrollo web:

- Arquitectura en capas (routes/controllers/services)
- Autenticacion JWT con control de roles
- Transacciones SQL para consistencia de inventario
- SPA con routing del lado cliente
- Degradacion graceful con mock offline en frontend
- Simulacion realista de pasarela de pagos y reembolsos
- Panel administrativo completo (CRUD, pedidos, reembolsos)

El codigo esta organizado para facilitar extension futura:
sustituir PaymentService por una pasarela real (Stripe, PayU),
migrar SQLite a PostgreSQL, o separar frontend en React/Vue.

--- Fin del informe ---

Generado automaticamente por scripts/generate_report_pdf.js